

Node.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
public class Node {
```

```
    int key;
```

```
    Node left;
```

```
    Node right;
```

```
    public Node(int key) {
```

```
        this.key = key;
```

```
        this.left = null;
```

```
        this.right = null;
```

```
    }
```

```
}
```

Lab1.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
import java.util.TreeSet;
```

```
public class Lab1 {
```

```
    public static void main(String[] args) {
```

```
        TreeSet<Integer> myset = new TreeSet<Integer>();
```

```
myset.add(25);
```

```
myset.add(20);
```

```
myset.add(30);
```

```
myset.add(15);
```

```
myset.add(23);
```

```
myset.add(28);
```

```
myset.add(35);
```

```
System.out.println(myset);
```

```
System.out.println("-----");
```

```
for(int x:myset)
```

```
    System.out.print(x+"\t");
```

```
System.out.println("\n-----");
```

```
System.out.println(myset.contains(28));
```

```
System.out.println(myset.contains(27));
```

```
myset.remove(15);
```

```
System.out.println(myset);
```

```
System.out.println("\n-----");
```

```
System.out.println(myset.first());
```

```
System.out.println(myset.last());
```

```
System.out.println(myset.lower(23));
```

```
System.out.println(myset.higher(28));
```

```
        System.out.println(myset.floor(24));  
        System.out.println(myset.ceiling(24));  
    }  
}
```

Lab2.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
import java.util.TreeMap;
```

```
public class Lab2 {
```

```
    public static void main(String[] args) {
```

```
        TreeMap<Integer,String> mymap = new  
        TreeMap<Integer,String>();
```

```
        mymap.put(25,"25");
```

```
        mymap.put(20,"20");
```

```
        mymap.put(30,"30");
```

```
        mymap.put(15,"15");
```

```
        mymap.put(23,"23");
```

```
        mymap.put(28,"28");
```

```
        mymap.put(35,"35");
```

```
System.out.println(mymap);  
System.out.println("-----");
```

```
System.out.println(mymap.containsKey(28));  
System.out.println(mymap.containsKey(27));
```

```
mymap.remove(15);  
System.out.println(mymap);  
System.out.println("-----");  
System.out.println(mymap.firstKey());  
System.out.println(mymap.lastKey());
```

```
System.out.println(mymap.ceilingKey(24));  
System.out.println(mymap.floorKey(24));  
System.out.println(mymap.lowerKey(25));  
System.out.println(mymap.higherKey(25));
```

```
System.out.println(mymap.ceilingEntry(24));  
System.out.println(mymap.floorEntry(24));  
System.out.println(mymap.lowerEntry(25));  
System.out.println(mymap.higherEntry(25));
```

```
}
```

```
}
```

Lab3.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Search in BST - Recursive Way
```

```
public class Lab3 {
```

```
    public static boolean search(Node myroot, int element) {
```

```
        if(myroot==null)
            return false;
```

```
        if(element==myroot.key)
            return true;
```

```
        else if(element<myroot.key)
            return search(myroot.left,element);
```

```
        else if(element>myroot.key)
            return search(myroot.right,element);
```

```
        return false;
```

```
    }
```

```
public static void main(String[] args) {  
  
    Node myroot1 = null;  
  
    boolean flag = search(myroot1, 25);  
    System.out.println(flag);  
  
    Node myroot2 = new Node(25);  
  
    flag = search(myroot2, 25);  
    System.out.println(flag);  
  
    Node myroot = new Node(25);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(10);  
    myroot.left.right = new Node(23);  
  
    myroot.right.left = new Node(27);  
    myroot.right.right = new Node(35);  
  
    flag = search(myroot, 27);  
    System.out.println(flag);  
}
```

```
    }  
}
```

Lab4.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
//Search in BST - Iterative Way
```

```
public class Lab4 {
```

```
    public static boolean search(Node myroot, int element) {
```

```
        while (myroot != null) {
```

```
            if (element == myroot.key)
```

```
                return true;
```

```
            else if (element < myroot.key)
```

```
                myroot = myroot.left;
```

```
            else if (element > myroot.key)
```

```
                myroot = myroot.right;
```

```
        }
```

```
        return false;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Node myroot1 = null;
```

```
    boolean flag = search(myroot1, 25);
```

```
    System.out.println(flag);
```

```
    Node myroot2 = new Node(25);
```

```
    flag = search(myroot2, 25);
```

```
    System.out.println(flag);
```

```
    Node myroot = new Node(25);
```

```
    myroot.left = new Node(20);
```

```
    myroot.right = new Node(30);
```

```
    myroot.left.left = new Node(10);
```

```
    myroot.left.right = new Node(23);
```

```
    myroot.right.left = new Node(27);
```

```
    myroot.right.right = new Node(35);
```

```
    flag = search(myroot, 27);
```



```

        System.out.println(flag);

        flag = search(myroot, 28);
        System.out.println(flag);

    }
}

```

Lab5.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Find Min in BST - Recursive Way
```

```
public class Lab5 {
```

```
    public static int findMin(Node myroot) {
```

```
        if(myroot==null)
```

```
            return -1;
```

```
        if(myroot.left==null)
```

```
            return myroot.key;
```

```
        else  
            return findMin(myroot.left);  
    }
```

```
public static void main(String[] args) {  
  
    Node myroot = new Node(25);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(10);  
    myroot.left.right = new Node(23);  
  
    myroot.right.left = new Node(27);  
    myroot.right.right = new Node(35);  
  
    myroot.left.left.left = new Node(6);  
  
    int min = findMin(myroot);  
    System.out.println(min);  
  
}
```

```
}
```

Lab6.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Find Min in BST - Iterative Way
```

```
public class Lab6 {
```

```
    public static int findMin(Node myroot) {
```

```
        if (myroot == null)
```

```
            return -1;
```

```
        Node current = myroot;
```

```
        while (current.left != null) {
```

```
            current = current.left;
```

```
        }
```

```
        return current.key;
```

```
    }
```

```
    public static void main(String[] args) {
```

```

        Node myroot = new Node(25);

        myroot.left = new Node(20);
        myroot.right = new Node(30);

        myroot.left.left = new Node(10);
        myroot.left.right = new Node(23);

        myroot.right.left = new Node(27);
        myroot.right.right = new Node(35);

        //myroot.left.left.left = new Node(6);

        int min = findMin(myroot);
        System.out.println(min);

    }
}

```

Lab7.java

.....

```
package com.jlcindia.binarysearchtrees;
```

// Find Max in BST - Recursive Way

```
public class Lab7 {  
  
    public static int findMax(Node myroot) {  
  
        if(myroot==null)  
            return -1;  
  
        if(myroot.right==null)  
            return myroot.key;  
        else  
            return findMax(myroot.right);  
    }  
}
```

```
public static void main(String[] args) {  
  
    Node myroot = new Node(25);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(10);  
    myroot.left.right = new Node(23);  
}
```

```

        myroot.right.left = new Node(27);
        myroot.right.right = new Node(35);

        myroot.right.right.right = new Node(66);

        int max = findMax(myroot);
        System.out.println(max);

    }
}

```

Lab8.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Find Max in BST - Iterative Way
```

```
public class Lab8 {
```

```
    public static int findMax(Node myroot) {
```

```
        if (myroot == null)
```

```
        return -1;
```

```
    Node current = myroot;
```

```
    while (current.right != null) {
```

```
        current = current.right;
```

```
    }
```

```
    return current.key;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Node myroot = new Node(25);
```

```
    myroot.left = new Node(20);
```

```
    myroot.right = new Node(30);
```

```
    myroot.left.left = new Node(10);
```

```
    myroot.left.right = new Node(23);
```

```
    myroot.right.left = new Node(27);
```

```
    myroot.right.right = new Node(35);
```

```
    myroot.right.right.right = new Node(66);
```

```
    int max = findMax(myroot);
```

```
    System.out.println(max);
```

```
    }  
}
```

Lab9.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Find Floor in BST - Iterative Way
```

```
//Equal or Largest in Smaller Keys
```

```
public class Lab9 {
```

```
    public static Node findFloor(Node myroot, int element) {
```

```
        Node result = new Node(-1);
```

```
        while (myroot != null) {
```

```
            if (element == myroot.key) {
```

```
                return myroot;
```

```
            } else if (element < myroot.key) {
```

```
                myroot = myroot.left;
```

```
            } else if (element > myroot.key) {
```

```
                result = myroot;
```

```
                myroot = myroot.right;
```

```
            }
```



```
    }  
    return result;  
}
```

```
public static void main(String[] args) {
```

```
    Node myroot = new Node(25);
```

```
    myroot.left = new Node(20);
```

```
    myroot.right = new Node(30);
```

```
    myroot.left.left = new Node(10);
```

```
    myroot.left.right = new Node(23);
```

```
    myroot.right.left = new Node(27);
```

```
    myroot.right.right = new Node(35);
```

```
    myroot.right.left.left = new Node(26);
```

```
    myroot.right.left.right = new Node(28);
```

```
    //myroot.left.left.left = new Node(9);
```

```
    Node floor = findFloor(myroot, 9);
```

```
    System.out.println(floor.key);
```

```
    }  
}
```

Lab10.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Find Ceil in BST - Iterative Way
```

```
// Equal or Smallest in Larger Keys
```

```
public class Lab10 {
```

```
    public static Node findCeil(Node myroot, int element) {
```

```
        Node result = myroot;
```

```
        while (myroot != null) {
```

```
            if (element == myroot.key) {
```

```
                return myroot;
```

```
            } else if (element < myroot.key) {
```

```
                result = myroot;
```

```
                myroot = myroot.left;
```

```
            } else if (element > myroot.key) {
```

```
                myroot = myroot.right;
```

```
        }  
    }  
    return result;  
}
```

```
public static void main(String[] args) {
```

```
    Node myroot = new Node(25);
```

```
    myroot.left = new Node(20);
```

```
    myroot.right = new Node(30);
```

```
    myroot.left.left = new Node(10);
```

```
    myroot.left.right = new Node(23);
```

```
    myroot.right.left = new Node(27);
```

```
    myroot.right.right = new Node(35);
```

```
    myroot.right.left.left = new Node(26);
```

```
    myroot.right.left.right = new Node(28);
```

```
    //myroot.left.left.left = new Node(9);
```

```
    Node ceil = findCeil(myroot, 29);
```

```
    System.out.println(ceil.key);
```

```
    }  
}
```

Lab11.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Insert Operation in BST - Recursive
```

```
public class Lab11 {
```

```
    public static void inorder(Node myroot) {  
        if (myroot != null) {  
            inorder(myroot.left);  
            System.out.println(myroot.key + "\t");  
            inorder(myroot.right);  
        }  
    }  
}
```

```
    public static Node insert(Node root, int element) {
```

```
        if (root == null) {  
            return new Node(element);
```

```

    }

    if(element<root.key) {
        root.left = insert(root.left,element);
    }else if(element>root.key) {
        root.right = insert(root.right,element);
    }

    return root;
}

```

```

public static void main(String[] args) {

```

```

    Node root = null;
    root = insert(root, 50);
    root = insert(root,25);
    root = insert(root,75);
    root = insert(root,30);
    root = insert(root,20);
    root = insert(root,60);
    root = insert(root,90);
    inorder(root);

```

```

    /*
    inorder(root);
    System.out.println("-----");

```

```

        root = insert(root,25);
        inorder(root);
        System.out.println("-----");
        root = insert(root,75);
        inorder(root);
        System.out.println("-----");
        root = insert(root,30);
        inorder(root);
        System.out.println("-----");
        root = insert(root,20);
        inorder(root);
        System.out.println("-----");
        root = insert(root,60);
        inorder(root);
        System.out.println("-----");
        root = insert(root,90);
        inorder(root);
        */
    }
}

```

Lab12.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Insert Operation in BST - Iterative
```

```
public class Lab12 {
```

```
    public static void inorder(Node myroot) {  
        if (myroot != null) {  
            inorder(myroot.left);  
            System.out.println(myroot.key + "\t");  
            inorder(myroot.right);  
        }  
    }
```

```
    public static Node insert(Node root, int element) {
```

```
        Node current = root;  
        Node parent = null;  
        while (current != null) {  
            parent = current;  
  
            if (element < current.key) {  
                current = current.left;  
            } else if (element > current.key) {  
                current = current.right;  
            }  
        }
```

```
        } else if (element == current.key) {  
            return root;  
        }  
    }  
}
```

```
Node temp = new Node(element);  
if(parent==null) {  
    return temp;  
}
```

```
if (element < parent.key)  
    parent.left = temp;  
else if(element > parent.key)  
    parent.right = temp;
```

```
return root;  
}
```

```
public static void main(String[] args) {
```

```
    Node root = null;  
    root = insert(root, 50);
```

```
    root = insert(root, 25);  
    root = insert(root, 75);  
    root = insert(root, 30);
```



```

        root = insert(root, 20);
        root = insert(root, 60);
        root = insert(root, 90);
        inorder(root);
        System.out.println("-----");
        root = insert(root, 10);
        inorder(root);
        System.out.println("-----");
        root = insert(root, 22);
        inorder(root);
        System.out.println("-----");
    }
}

```

Lab13.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Delete Operation in BST - Recursive
```

```
public class Lab13 {
```

```
public static Node findMin(Node myroot) {  
  
    if(myroot==null)  
        return new Node(-1);  
  
    if(myroot.left==null)  
        return myroot;  
    else  
        return findMin(myroot.left);  
}
```

```
public static void inorder(Node myroot) {  
    if (myroot != null) {  
        inorder(myroot.left);  
        System.out.println(myroot.key + "\t");  
        inorder(myroot.right);  
    }  
}
```

```
public static Node delete(Node root, int element) {  
  
    if(root==null) {  
        return root;  
    }  
}
```

```

if(element<root.key) {
    root.left = delete(root.left,element);
}else if(element>root.key) {
    root.right = delete(root.right,element);
}else if(element == root.key) {
    //Element Found to Delete
    if(root.left==null && root.right==null) {
        //1. No Children
        root=null;
        return root;
    }else if(root.right==null) {
        //2. One Left Child
        root = root.left;
        return root;
    } else if(root.left==null) {
        //3. One Right Child
        root = root.right;
        return root;
    }else {
        //4. Two Childen
        Node min = findMin(root.right);
        root.key = min.key;
        root.right = delete(root.right,min.key);
        return root;
    }
}

```

```
        return root;
    }
```

```
public static void main(String[] args) {
```

```
    Node myroot = new Node(50);
```

```
    myroot.left = new Node(25);
```

```
    myroot.right = new Node(75);
```

```
    myroot.left.left = new Node(20);
```

```
    myroot.left.right = new Node(30);
```

```
    myroot.right.left = new Node(60);
```

```
    myroot.right.right = new Node(90);
```

```
    myroot.right.right.left = new Node(85);
```

```
    myroot.right.right.right = new Node(95);
```

```
    myroot.right.right.left.right = new Node(87);
```

```
    myroot.right.right.left.left = new Node(80);
```

```
    inorder(myroot);
```

```
    myroot = delete(myroot,75);
```

```
    System.out.println("-----");
```

```
    inorder(myroot);
```

```
    }  
}
```

Lab14.java

.....

```
package com.jlcindia.binarysearchtrees;
```

```
// Check whether Binary Tree is BST or Not
```

```
public class Lab14 {  
    public static boolean isBST(Node root,int min,int max) {  
        if(root==null)  
            return true;  
  
        return (root.key>min &&  
                root.key<max &&  
                isBST(root.left, min, root.key) &&  
                isBST(root.right, root.key, max));  
    }  
  
    public static void main(String[] args) {
```

```
Node myroot = new Node(50);
```

```
myroot.left = new Node(25);
```

```
myroot.right = new Node(75);
```

```
myroot.left.left = new Node(20);
```

```
myroot.left.right = new Node(30);
```

```
myroot.right.left = new Node(60);
```

```
myroot.right.right = new Node(90);
```

```
boolean flag
```

```
=isBST(myroot,Integer.MIN_VALUE,Integer.MAX_VALUE);
```

```
System.out.println(flag);
```

```
}
```

```
}
```